

ALU-Forge'26

DESIGN AND IMPLEMENTATION OF AN 8-BIT ALU

Using Registers, Common Bus, Clock and Discrete Digital ICs

Team Name: Bit-Mechanix

Team Members: Mukesh S

Harikrishnan R

Deshni A

1. Abstract:

This project implements an 8-bit Arithmetic Logic Unit (ALU) using discrete digital logic ICs and a shared 8-bit bus architecture. Operands set via DIP switches are sequentially loaded into Registers A and B using push-button controls and a 1 Hz clock source. The operands are processed through an 8-bit adder/subtractor circuit—where addition or subtraction is determined by the carry-in control of the first adder stage.

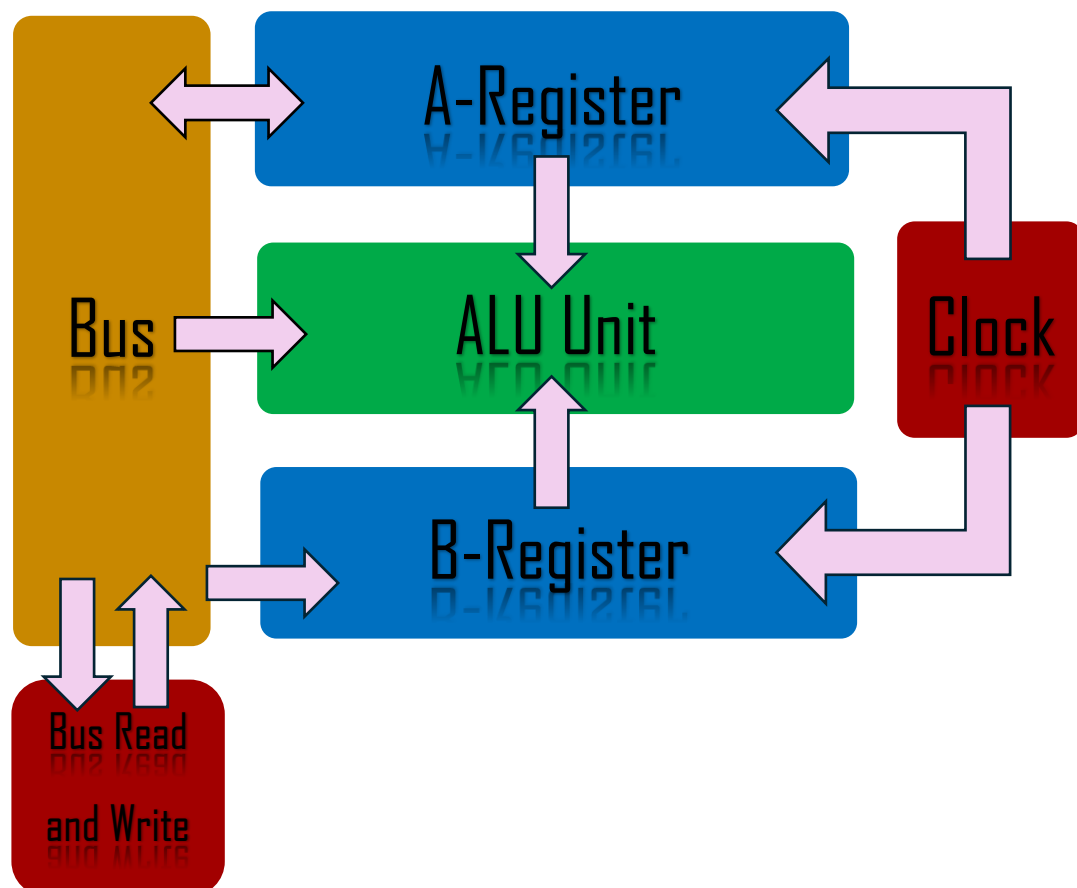
2. Objectives:

- ✓ Design & Arithmetic Processing: Construct an 8-bit ALU to perform addition and subtraction.
- ✓ Storage & Data Transfer: Implement dedicated A/B registers and a common 8-bit bus with 74HC245 transceiver control for bidirectional data routing between Register A and the Bus.
- ✓ Control & Input Interface: Utilize DIP switches for operand input, push buttons for manual register/bus control, and a 1 Hz clock for step-by-step observation.
- ✓ Hardware Demonstration: Verify continuous arithmetic and register-transfer operations at the hardware level.

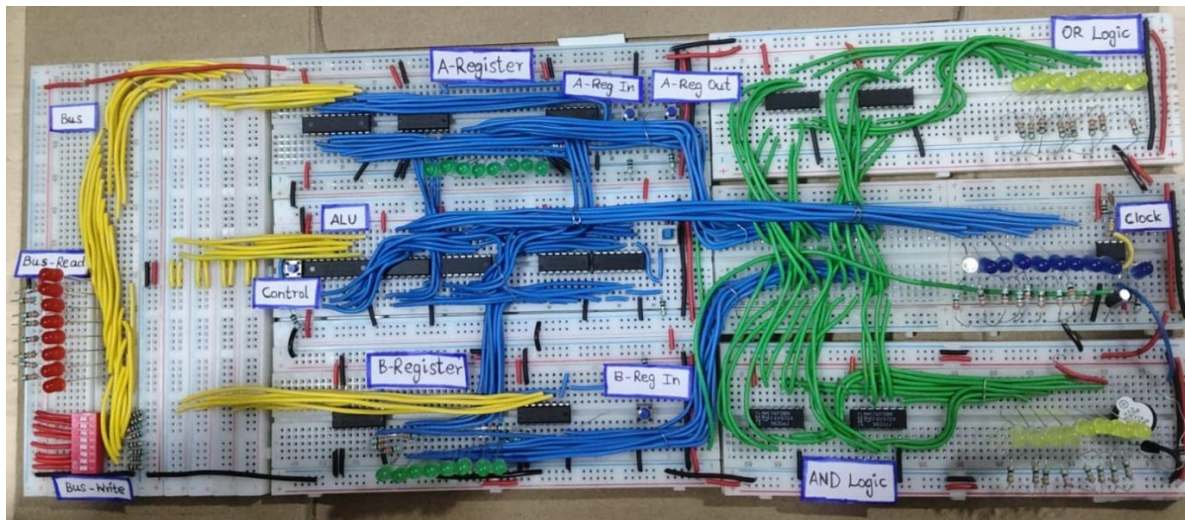
3. System Overview:

The developed system follows a register-bus-arithmetic architecture. An 8-bit value selected using the DIP switches is placed on the common bus. When the A-register enable signal is activated, the value present on the bus is captured by Register A on the appropriate clock event. Register B is loaded using the same principle. The outputs of the two registers are connected to the arithmetic section. The arithmetic section performs addition or subtraction according to the selected control condition. The A-register output and the 74HC245 transceiver provide controlled access to the bus, allowing data to be transferred through the shared data path.

4. Block Diagram:



5. Designed Circuit:



6. Overall Working Procedure:

- ✓ Load Operand A: Set the desired 8-bit value on the DIP switches, place it on the bus, enable A Register, and applied at the appropriate clock event.
- ✓ Store A: The A Register captures the bus value and retains it after the input switches are changed.
- ✓ Load Operand B: Change the DIP switches to the second operand, place the value on the bus, enable B Register, and applied at the appropriate clock event.
- ✓ Store B: The B Register captures and retains the second operand.
- ✓ Select Operation: Apply the appropriate operation-control/carry-in condition for addition or subtraction to be performed in the ALU Unit.
- ✓ Generate Result: The cascaded adder stages process the two operands and produce the 8-bit result and displays at the ALU Unit.
- ✓ Transfer Result: Use the bus/transceiver and register controls as required by the implemented data path to transfer or reuse the result.

7. A-Register:

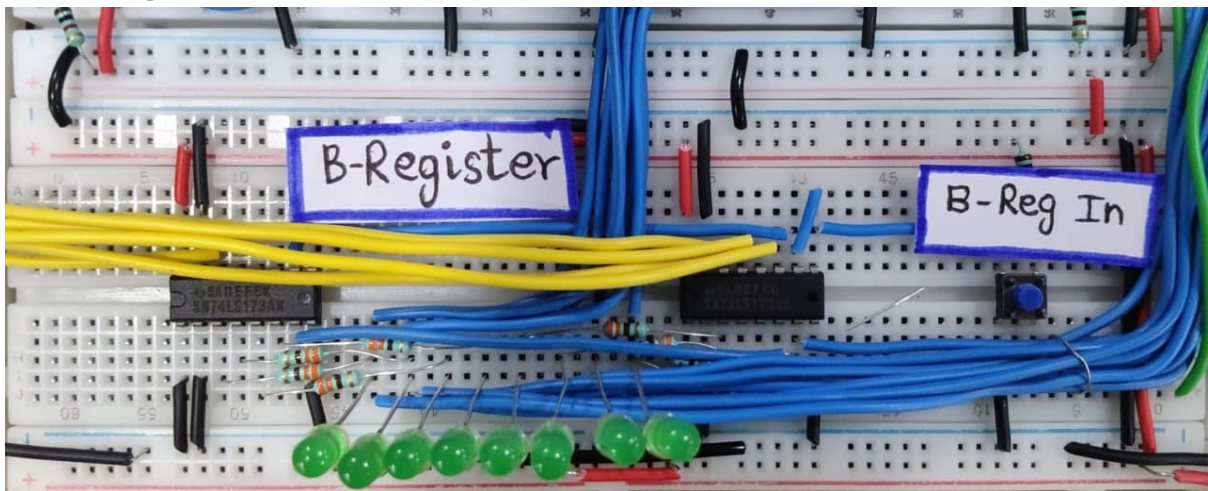


Register A stores the first operand. It receives the 8-bit value from the common bus when its enable control is activated. The A-Reg Out function allows the stored value to be made available on the bus when the corresponding output control is enabled. It is also called as an Accumulator as it accumulates the results of the Arithmetic Operations at the required times.

Components List:

Component	Quantity	Functionality
IC 74173	2	8-bit Data Storage (D-FF)
IC 74245	1	Bidirectional Tri-state buffer
Push Button	2	Write and Read enable (Active Low)
LED (Green)	8	Data Representation
Resistor (330 ohm)	8	Power Limiting to LED
Resistor (1k ohm)	2	Pull Up Resistor

8. B-Register:

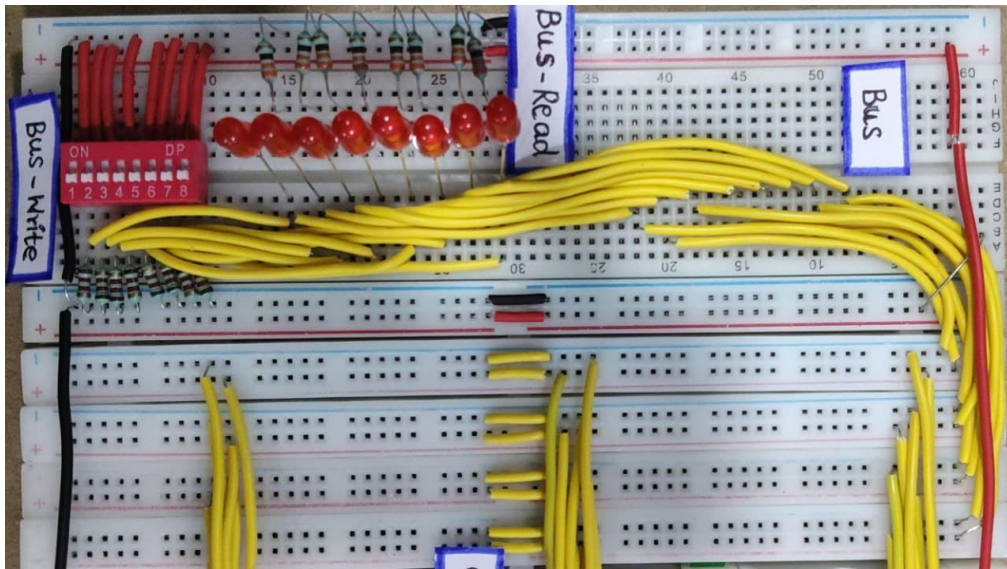


Register B stores the second operand. It is loaded from the same common bus using its own enable control. Keeping the operands in separate registers allows the arithmetic section to receive two stable operand values.

Components List:

Component	Quantity	Functionality
IC74173	2	8-bit Data Storage (D-ff)
Push Button	1	Write enable (Active Low)
LED (Green)	8	Data Representation
Resistor (330 ohm)	8	Power Limiting to LED
Resistor (1k ohm)	1	Pull Up Resistor

9. Common 8-bit Bus:



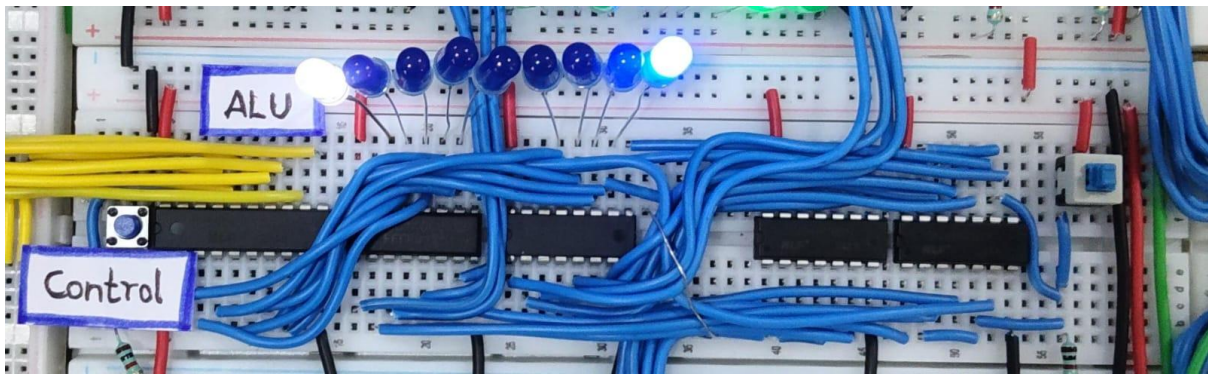
The common bus is the shared 8-bit data path of the system. Instead of requiring separate direct connections between every pair of modules, the bus provides a common route through which data can be transferred. A register places data on the bus only when its output is enabled, while a receiving register captures the bus value when its input/enable control is active.

Components List:

Component	Quantity	Functionality
DIP Switch (8-bit)	1	Write BUS Data (Active High)
LED (Red)	8	Data Representation
Resistor (330 ohm)	8	Power Limiting to LED
Resistor (1.2k ohm)	8	Pull Down Resistor

Bus contention must be avoided: two independent outputs should not simultaneously drive the same bus unless the circuit has been specifically designed for that condition.

10. The ALU Unit:



The arithmetic unit takes the 8-bit values held in Register A and Register B and produces an 8-bit result. It is built from cascaded adder stages, so the carry from each bit position ripples into the next higher-order bit.

Operation is selected using a single control line, taken as the carry-in (C_{in}) of the first adder stage. This same C_{in} line is also fed to a bank of XOR gates on the B inputs, so it controls both the 1's complement of B and the +1 needed to complete the 2's complement:

Addition ($C_{in} = 0$): XOR gates pass B unchanged $\rightarrow$ Result = $A + B$

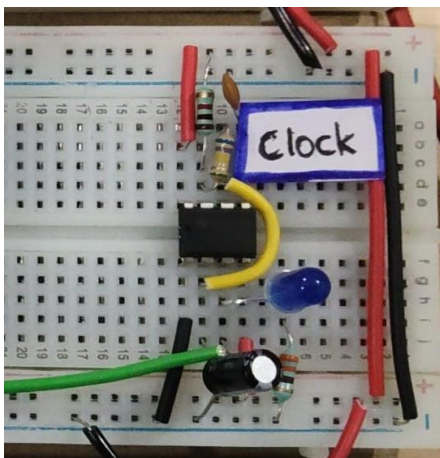
Subtraction ($C_{in} = 1$): XOR gates invert B (1's complement) and C_{in} adds the final 1 $\rightarrow$ Result = $A + (\sim B) + 1 = A - B$

Components List:

Component	Quantity	Functionality
IC 74008	2	8-bit Arithmetic Operation
IC 74245	1	Bidirectional Tri-state buffer
IC 74286	2	2's complement B Register Data for SUB Operation
Push Button	1	Read enable (Active Low)
DPDT Switch	1	Operation Selection (ADD = 0, SUB = 1)
LED (Blue)	8	Data Representation
LED (Yellow)	1	Carry Out Representation

Component	Quantity	Functionality
Resistor (330 ohm)	9	Power Limiting to LED
Resistor (1k ohm)	1	Pull Up Resistor

11. Clock:



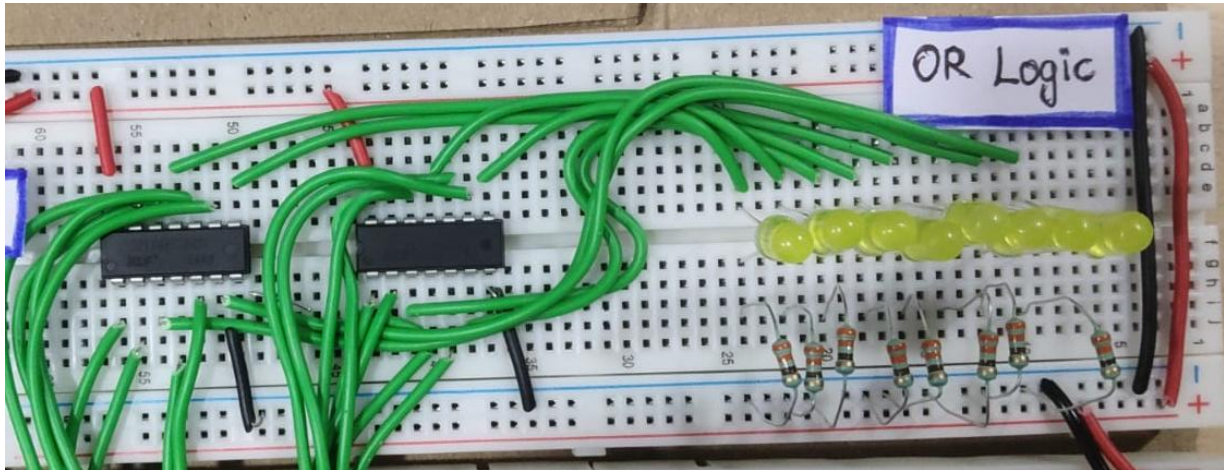
The 1 Hz clock serves as a predictable timing reference to synchronize sequential operations—such as loading data into registers and transferring signals across the data path—without performing any arithmetic itself. Operating at one cycle per second slows down state changes, making data movement and register loading clearly visible during breadboard debugging and hardware demonstrations.

Components List:

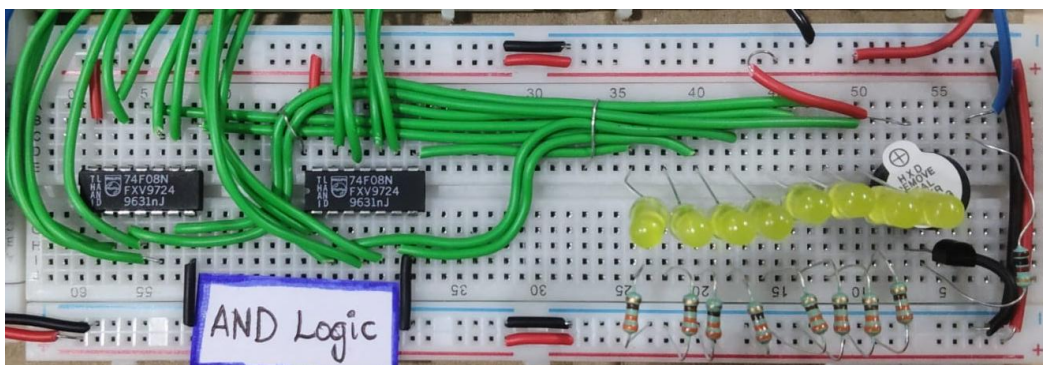
Component	Quantity	Functionality
IC555	1	Timer IC
LED (Blue)	1	CLK Representation
Resistor (330 ohm)	1	Power Limiting to LED
Resistor (1k ohm)	1	Frequency Control
Resistor (680k ohm)	1	Frequency Control
Capacitor (1uF, 0.1uF)	1, 1	Frequency Control

12. Additional Designs:

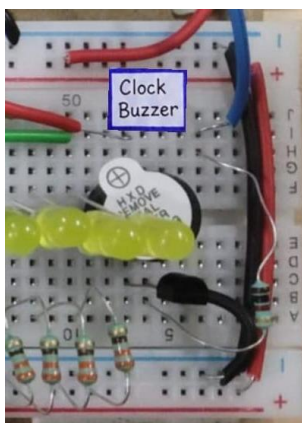
OR Logic: This block performs the bit-wise logical OR operation of the 2 register outputs.



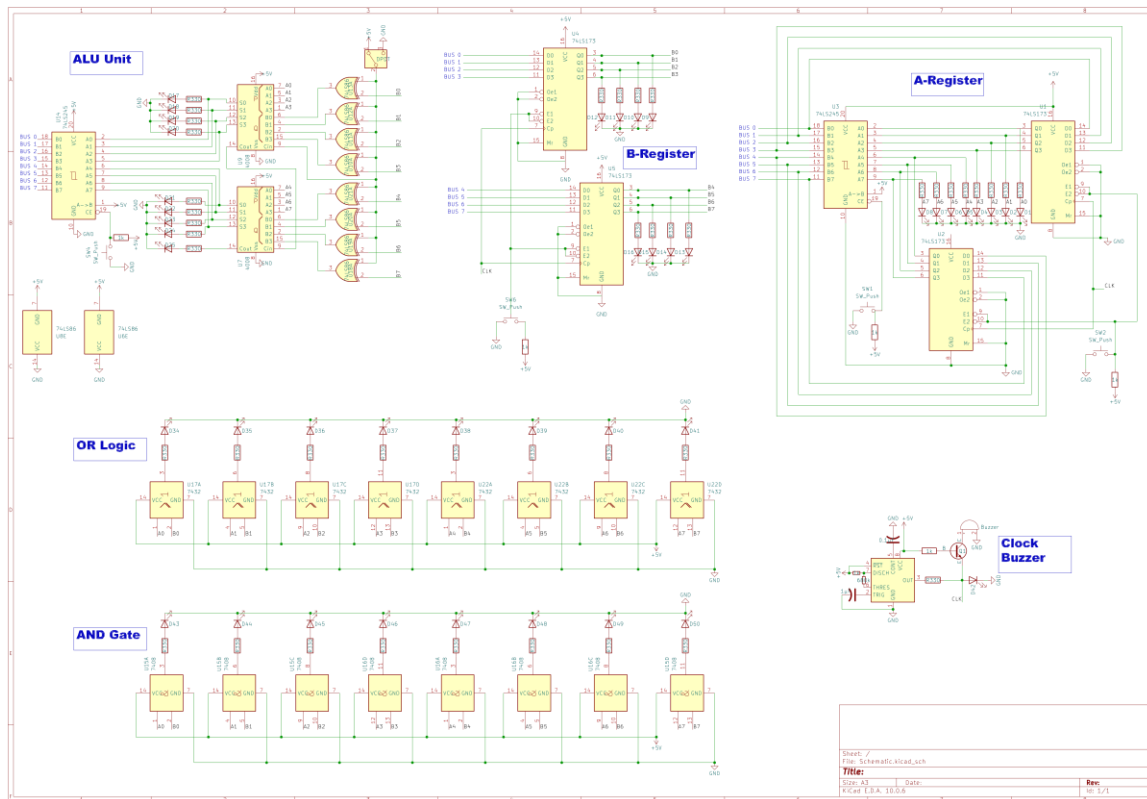
AND Logic: This block performs the bit-wise logical AND operation of the 2 register outputs.



Clock Buzzer: The here is used to indicate the clock pulse as a sound.



13. Schematic Design:



14. Reference:

- Website Link: <https://eater.net/8bit/alu>
- YouTube Video Link:
https://youtu.be/CiMaWbz_6E8?si=DILSAmiYJqOuL-hG